Genetic Algorithms

Artificial Intelligence — Computer Science — Sofia University

Bogomil Stoyanov

© 2026 Bogomil Stoyanov

Outline

1. The idea

2. Selection

3. Crossover

4. A worked population

5. Summary

© 2026 Bogomil Stoyanov

The idea

© 2026 Bogomil Stoyanov

Stochastic beam search with recombination

Recall beam search from session 03: keep ℓ states, generate their successors, keep the best ℓ. It got stuck because all ℓ states tend to end up in the same region.

© 2026 Bogomil Stoyanov

Stochastic beam search with recombination

Recall beam search from session 03: keep ℓ states, generate their successors, keep the best ℓ. It got stuck because all ℓ states tend to end up in the same region.

Stochastic beam search fixes that by choosing which to keep *probabilistically*, weighted by quality, instead of always taking the top ℓ.

© 2026 Bogomil Stoyanov

Stochastic beam search with recombination

Recall beam search from session 03: keep ℓ states, generate their successors, keep the best ℓ. It got stuck because all ℓ states tend to end up in the same region.

Stochastic beam search fixes that by choosing which to keep *probabilistically*, weighted by quality, instead of always taking the top ℓ.

A **genetic algorithm** adds one more thing: successors are generated from **pairs** of states rather than one at a time.

© 2026 Bogomil Stoyanov

Stochastic beam search with recombination

Recall beam search from session 03: keep ℓ states, generate their successors, keep the best ℓ. It got stuck because all ℓ states tend to end up in the same region.

Stochastic beam search fixes that by choosing which to keep *probabilistically*, weighted by quality, instead of always taking the top ℓ.

A **genetic algorithm** adds one more thing: successors are generated from **pairs** of states rather than one at a time.

That is the conceptual content. The biological vocabulary (population, chromosome, gene, fitness, crossover, mutation) is a naming convention laid over stochastic beam search with recombination.

© 2026 Bogomil Stoyanov

Stochastic beam search with recombination

Recall beam search from session 03: keep ℓ states, generate their successors, keep the best ℓ. It got stuck because all ℓ states tend to end up in the same region.

Stochastic beam search fixes that by choosing which to keep *probabilistically*, weighted by quality, instead of always taking the top ℓ.

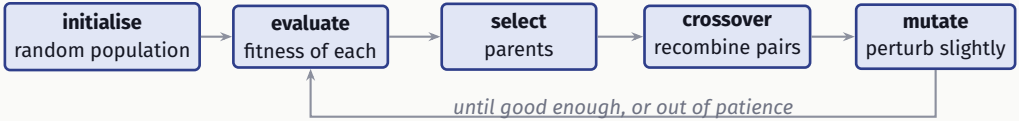
A **genetic algorithm** adds one more thing: successors are generated from **pairs** of states rather than one at a time.

That is the conceptual content. The biological vocabulary (population, chromosome, gene, fitness, crossover, mutation) is a naming convention laid over stochastic beam search with recombination.

And the analogy should not be pushed. Real genes encode the machinery of their own replication; here they encode a candidate answer and nothing more.

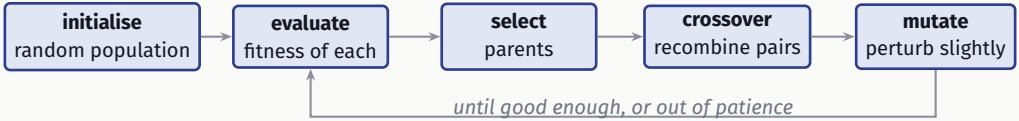
© 2026 Bogomil Stoyanov

The loop



© 2026 Bogomil Stoyanov

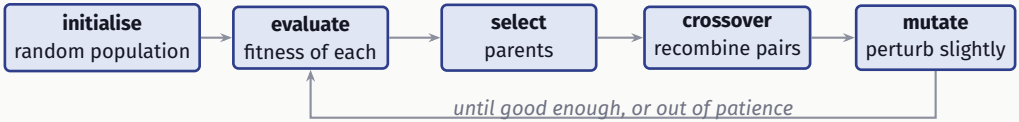
The loop



Selection supplies the pressure towards quality. **Crossover** combines partial solutions. **Mutation** keeps diversity alive so the population does not collapse to one point.

© 2026 Bogomil Stoyanov

The loop



Selection supplies the pressure towards quality. **Crossover** combines partial solutions. **Mutation** keeps diversity alive so the population does not collapse to one point.

There is a hard prerequisite: a state must be **encoded as a string** whose pieces are meaningful independently. If chopping a candidate in half and splicing it to another half is meaningless, crossover is just noise and the GA degenerates to random search.

© 2026 Bogomil Stoyanov

The algorithm

GENETIC-ALGORITHM(PROBLEM)

```
1  P ← random population of size N
2  repeat
3      for each x in P: evaluate FITNESS(x)
4      P′ ← ∅
5      while |P′| < N
6          x, y ← SELECT(P)              ▷ fitness-weighted
7          c ← CROSSOVER(x, y)
8          with small probability: MUTATE(c)
9          add c to P′
10     P ← P′
11 until satisfied
12 return the fittest member of P
```

© 2026 Bogomil Stoyanov

The algorithm

GENETIC-ALGORITHM(PROBLEM)

```
1  P ← random population of size N
2  repeat
3      for each x in P: evaluate FITNESS(x)
4      P′ ← ∅
5      while |P′| < N
6          x, y ← SELECT(P)              ▷ fitness-weighted
7          c ← CROSSOVER(x, y)
8          with small probability: MUTATE(c)
9          add c to P′
10     P ← P′
11 until satisfied
12 return the fittest member of P
```

Four things you choose, and they matter far more than the loop:

- the **encoding** (line 1);
- the **fitness** function (line 3);
- the **selection** scheme (line 6);
- the **crossover** operator (line 7).

© 2026 Bogomil Stoyanov

The algorithm

GENETIC-ALGORITHM(PROBLEM)

```
1  P ← random population of size N
2  repeat
3      for each x in P: evaluate FITNESS(x)
4      P′ ← ∅
5      while |P′| < N
6          x, y ← SELECT(P)              ▷ fitness-weighted
7          c ← CROSSOVER(x, y)
8          with small probability: MUTATE(c)
9          add c to P′
10     P ← P′
11 until satisfied
12 return the fittest member of P
```

Four things you choose, and they matter far more than the loop:

- the **encoding** (line 1);
- the **fitness** function (line 3);
- the **selection** scheme (line 6);
- the **crossover** operator (line 7).

The rest of this session is those four choices. The loop itself is never the problem.

© 2026 Bogomil Stoyanov

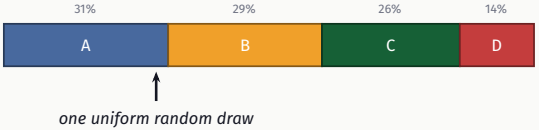
Selection

© 2026 Bogomil Stoyanov

Roulette-wheel selection

Give each individual a slice of the wheel proportional to its fitness, then spin:

$$P(x_i) = \frac{f(x_i)}{\sum_j f(x_j)}$$

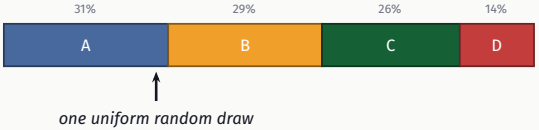


© 2026 Bogomil Stoyanov

Roulette-wheel selection

Give each individual a slice of the wheel proportional to its fitness, then spin:

$$P(x_i) = \frac{f(x_i)}{\sum_j f(x_j)}$$



Three real problems:

- fitness must be **positive**: a negative or zero objective has no meaningful slice;
- one very fit individual takes most of the wheel and the population collapses onto it (**premature convergence**);
- when everyone is nearly equally fit, the slices are nearly equal and there is *no* selection pressure left.

It is sensitive to the *scale* of the fitness values, not just their order.

© 2026 Bogomil Stoyanov

Other selection schemes

Scheme	How	Why
Tournament	pick k at random, keep the best	pressure tuned by k; ignores fitness *scale*
Rank-based	slice by rank, not by fitness	immune to outliers and to scale
Stochastic universal sampling	one random offset, then N evenly spaced pointers	same expectation as roulette, far less variance
Elitism	copy the best few through unchanged	the best answer can never be lost
Truncation	breed only from the top 20–50%	brutally simple; converges fast and early

© 2026 Bogomil Stoyanov

Other selection schemes

Scheme	How	Why
Tournament	pick k at random, keep the best	pressure tuned by k; ignores fitness *scale*
Rank-based	slice by rank, not by fitness	immune to outliers and to scale
Stochastic universal sampling	one random offset, then N evenly spaced pointers	same expectation as roulette, far less variance
Elitism	copy the best few through unchanged	the best answer can never be lost
Truncation	breed only from the top 20–50%	brutally simple; converges fast and early

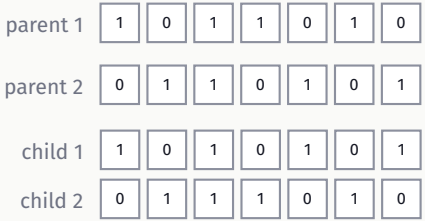
Tournament with $k = 2$ or 3 is the sane default: cheap, scale-free, one tunable knob. Add **elitism**: without it a GA can get *worse* from one generation to the next.

© 2026 Bogomil Stoyanov

Crossover

© 2026 Bogomil Stoyanov

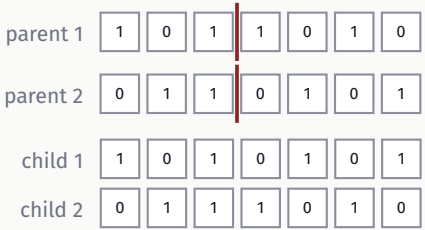
One-point crossover



Two binary-coded parents.

© 2026 Bogomil Stoyanov

One-point crossover



Choose one cut point at random, here after gene 3.

© 2026 Bogomil Stoyanov

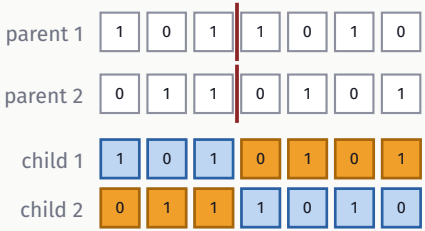
One-point crossover



Child 1 takes the head of parent 1…

© 2026 Bogomil Stoyanov

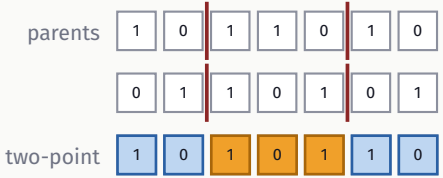
One-point crossover



…and the tail of parent 2. Child 2 is the complement. **Each child inherits a contiguous block from each parent**, which is the point: blocks that work together stay together.

© 2026 Bogomil Stoyanov

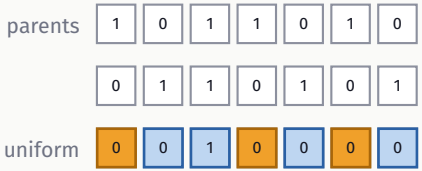
Two-point and uniform



Two-point: swap the middle segment instead of the tail. It can keep together blocks that sit at both ends of the string, which one-point always splits.

© 2026 Bogomil Stoyanov

Two-point and uniform

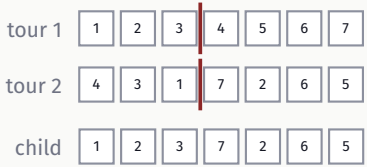


Uniform: decide *per gene*, by coin flip, which parent it comes from. Maximum mixing, and no notion of a block at all, so it destroys exactly the structure the other two try to preserve.

© 2026 Bogomil Stoyanov

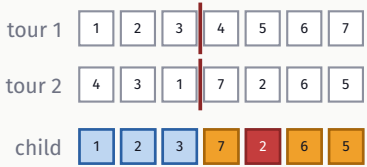
Now try that on a permutation

A tour of 7 cities is a *permutation*: every city exactly once.



© 2026 Bogomil Stoyanov

Now try that on a permutation

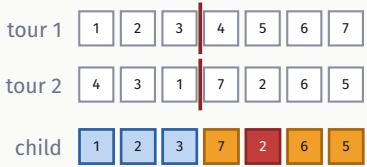


A tour of 7 cities is a *permutation*: every city exactly once.

One-point crossover gives 1 2 3 7 2 6 5, with city **2 twice**, city 4 never. **Not a tour.**

© 2026 Bogomil Stoyanov

Now try that on a permutation



A tour of 7 cities is a *permutation*: every city exactly once.

One-point crossover gives 1 2 3 7 2 6 5, with city **2 twice**, city 4 never. **Not a tour.**

> The binary operators assume genes are *independent*. In a permutation they are maximally dependent: which value a position can hold depends on every other position. That is why order-based crossovers exist.

© 2026 Bogomil Stoyanov

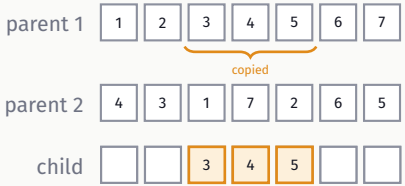
Partially-mapped crossover (PMX)



Pick a segment: positions 3 to 5.

© 2026 Bogomil Stoyanov

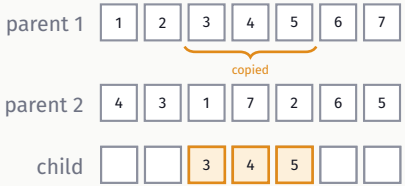
Partially-mapped crossover (PMX)



Copy it from parent 1 into the child: $3, 4, 5$.

© 2026 Bogomil Stoyanov

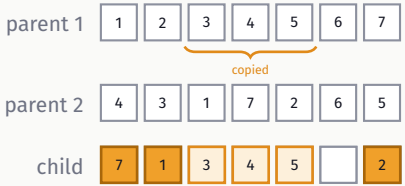
Partially-mapped crossover (PMX)



Parent 2's segment holds $1, 7, 2$, which the child still needs but cannot place at those positions.

© 2026 Bogomil Stoyanov

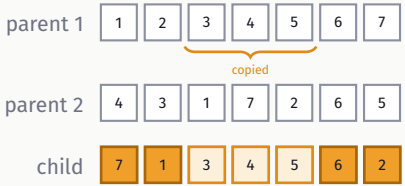
Partially-mapped crossover (PMX)



So each is placed where *its partner* sits in parent 2: 1 → position 2, 7 → position 1, 2 → position 7.

© 2026 Bogomil Stoyanov

Partially-mapped crossover (PMX)



Fill what is left from parent 2. Child = 7 1 3 4 5 6 2, **a valid permutation**, and it inherited a real block of parent 1's tour.

© 2026 Bogomil Stoyanov

Cycle crossover (CX)



© 2026 Bogomil Stoyanov

Cycle crossover (CX)



Follow a **cycle**: position 1 holds 8 in P_1 and 0 in P_2; 0 sits at position 10 in P_1, which holds 9 in P_2; 9 sits at position 9. Position 9 holds 8 in P_2, back where we started.

© 2026 Bogomil Stoyanov

Cycle crossover (CX)



Follow a **cycle**: position 1 holds 8 in P_1 and 0 in P_2; 0 sits at position 10 in P_1, which holds 9 in P_2; 9 sits at position 9. Position 9 holds 8 in P_2, back where we started.

Child 1 takes P_1 at the cycle's positions $\{1, 9, 10\}$ and P_2 everywhere else. Child 2 is the mirror image.

© 2026 Bogomil Stoyanov

Cycle crossover (CX)



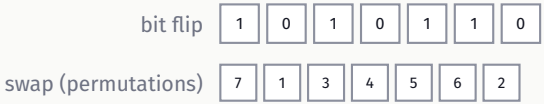
Follow a **cycle**: position 1 holds 8 in P_1 and 0 in P_2; 0 sits at position 10 in P_1, which holds 9 in P_2; 9 sits at position 9. Position 9 holds 8 in P_2, back where we started.

Child 1 takes P_1 at the cycle's positions $\{1, 9, 10\}$ and P_2 everywhere else. Child 2 is the mirror image.

> Every position keeps a value one of its parents had there , so CX preserves *absolute position*, where PMX preserves *relative order*. Which you want depends on the problem.

© 2026 Bogomil Stoyanov

Mutation



© 2026 Bogomil Stoyanov

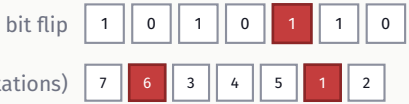
Mutation



A bit flip is fine for binary codes. For a permutation it would break validity, so mutate by *swapping* two positions instead: the operator has to respect the encoding, exactly as crossover does.

© 2026 Bogomil Stoyanov

Mutation



A bit flip is fine for binary codes. For a permutation it would break validity, so mutate by *swapping* two positions instead: the operator has to respect the encoding, exactly as crossover does.

Crossover can only recombine values already in the population. Once every individual agrees on a gene, no crossover will ever change it. Mutation is the *only* way to reintroduce a lost value, which is why a rate of zero fails, and why a rate that is too high turns the search into a random walk.

© 2026 Bogomil Stoyanov

A worked population

© 2026 Bogomil Stoyanov

8-queens, one generation

	Encoding	Fitness	Share	Selected?
x_1	`24748552`	24	31%	twice
x_2	`32752411`	23	29%	once
x_3	`24415124`	20	26%	once
x_4	`32543213`	11	14%	not at all

Encoding. Eight digits; digit *i* is the row of the queen in column *i*. One queen per column is built into the representation, so those constraints can never be violated.

© 2026 Bogomil Stoyanov

8-queens, one generation

	Encoding	Fitness	Share	Selected?
x_1	`24748552`	24	31%	twice
x_2	`32752411`	23	29%	once
x_3	`24415124`	20	26%	once
x_4	`32543213`	11	14%	not at all

Encoding. Eight digits; digit *i* is the row of the queen in column *i*. One queen per column is built into the representation, so those constraints can never be violated.

Fitness. The number of *non-attacking* pairs, out of $\binom{8}{2} = 28$. A solution scores 28.

© 2026 Bogomil Stoyanov

8-queens, one generation

	Encoding	Fitness	Share	Selected?
x_1	24748552	24	31%	twice
x_2	32752411	23	29%	once
x_3	24415124	20	26%	once
x_4	32543213	11	14%	not at all

Encoding. Eight digits; digit *i* is the row of the queen in column *i*. One queen per column is built into the representation, so those constraints can never be violated.

Fitness. The number of *non-attacking* pairs, out of $\binom{8}{2} = 28$. A solution scores 28.

Fitness counts what is *right*, not what is wrong: roulette needs a positive quantity to slice up, and 28 − conflicts gives one while ranking states identically. x_4 at 14% is unlikely to breed, but not forbidden, which is the whole point of *stochastic* selection.

© 2026 Bogomil Stoyanov

Encoding TSP and knapsack

Travelling salesman

Encoding: a permutation of the n cities.

Fitness: 1/tour length, or const – length.

Crossover: must be order-based: PMX, CX, OX.

Mutation: swap two cities, or reverse a sub-tour.

© 2026 Bogomil Stoyanov

Encoding TSP and knapsack

Travelling salesman

Encoding: a permutation of the n cities.

Fitness: 1/tour length, or const – length.

Crossover: must be order-based: PMX, CX, OX.

Mutation: swap two cities, or reverse a sub-tour.

0/1 knapsack

Encoding: a bit per item: take it or leave it.

Fitness: total value, or 0 if over the weight limit.

Crossover: one-point or uniform both work.

Mutation: flip a bit.

© 2026 Bogomil Stoyanov

Encoding TSP and knapsack

Travelling salesman

Encoding: a permutation of the n cities.

Fitness: 1/tour length, or const – length.

Crossover: must be order-based: PMX, CX, OX.

Mutation: swap two cities, or reverse a sub-tour.

0/1 knapsack

Encoding: a bit per item: take it or leave it.

Fitness: total value, or 0 if over the weight limit.

Crossover: one-point or uniform both work.

Mutation: flip a bit.

Knapsack shows the other half of the design problem: what to do with an **infeasible** child. Score it zero and you throw away information; repair it and you bias the search; penalise it in proportion to the overweight and you usually get the best of both. This choice is as consequential as the choice of operator.

© 2026 Bogomil Stoyanov

Summary

© 2026 Bogomil Stoyanov

Reasonable and unreasonable uses

Reasonable

- the space is huge and badly understood;
- no usable gradient, no admissible heuristic;
- a good answer now beats the best answer eventually;
- candidates split into parts meaningful on their own.

© 2026 Bogomil Stoyanov

Reasonable and unreasonable uses

Reasonable

- the space is huge and badly understood;
- no usable gradient, no admissible heuristic;
- a good answer now beats the best answer eventually;
- candidates split into parts meaningful on their own.

- **No optimality guarantee, ever**, not even an approximation bound.
- Many sensitive parameters: population size, rates, tournament size.
- If an admissible heuristic exists, A^* wins and *proves* its answer.
- With no meaningful blocks, crossover adds nothing over random restarts.

© 2026 Bogomil Stoyanov

Reasonable and unreasonable uses

Reasonable

- the space is huge and badly understood;
- no usable gradient, no admissible heuristic;
- a good answer now beats the best answer eventually;
- candidates split into parts meaningful on their own.

- **No optimality guarantee, ever**, not even an approximation bound.
- Many sensitive parameters: population size, rates, tournament size.
- If an admissible heuristic exists, A^* wins and *proves* its answer.
- With no meaningful blocks, crossover adds nothing over random restarts.

The encoding and the fitness function decide whether a GA works at all. Get those wrong and tuning the loop will not rescue it.

© 2026 Bogomil Stoyanov